МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

Звіт з практичної роботи 1

з дисципліни «Розробка програмного забезпечення мобільних пристроїв»

Практичні заняття 1–4

Варіант №1

Система прогнозування споживання електроенергії

Просунутий рівень

Виконала студентка групи ТВ-з31

Вахрушева Христина Костянтинівна

https://github.com/Christ1naa/RPZMP

Перевірив: Олексій Леонідович Недашківський

Київ — 2026

1 Завдання та відповідність вимогам

Тема роботи — створення Android-застосунку EnergyForecast мовою Kotlin із використанням Jetpack Compose. Мета — опанувати декларативний інтерфейс, керування станом, організацію Gradle-проєкту, відокремлення обчислень від UI та побудову форм авторизації.

Індивідуальне завдання — варіант №1 «Система прогнозування споживання електроенергії». Приклад сонячної електростанції в методичці належить варіанту №30. У цій роботі його навчальні прийоми адаптовано до оцінювання енергії, яку споживають групи електроприладів, без моделювання сонячної генерації [1].

Потрібно реалізувати початковий екран із динамічними показниками й анімацією; організувати Gradle та бізнес-логіку; створити компоненти для введення, зображень і взаємодії; додати вхід, реєстрацію, валідацію та стан обробки.

Таблиця 1 — Відповідність вимогам практичних занять

Вимога	Реалізація та розділ звіту
Заняття 1, с. 11–12: State, оновлення, Material, анімація	DemoDashboard.kt; розділ 3
Заняття 2, с. 20–21: окрема логіка, Gradle, стороння візуалізація	logic, model, MPAndroidChart; розділи 2, 4
Заняття 3, с. 30–31: текст, зображення, кнопки, адаптивний UI	DeviceCard, DeviceEditorDialog, LazyColumn; розділ 5
Заняття 4, с. 40–41: форми, помилки, loading, компоненти	AuthScreen, AuthFields, AuthViewModel; розділ 7
Пояснення UI, стану, формул і валідації	Український KDoc, уривки коду; розділи 3–8
Вказівки викладача	Висновок; проєкт у GitHub; посилання під ПІБ на титулі
Звіт і підготовка до захисту	Цей звіт; окремий аркуш відповідей на контрольні питання

Версія проєкту: гілка main, commit ebd9a8524d61c1241601acd0162c3221b9bd8bbe. Дата перевірки — 29.10.2026. У цій версії змерджено PR №1 з українськими поясненнями та граничними тестами.

2 Теоретичні відомості та середовище

Kotlin — статично типізована мова; моделі data class зберігають параметри предметної області [2, 3]. Jetpack Compose описує інтерфейс через @Composable-функції. Коли змінюється спостережуваний стан, Compose повторно виконує залежні ділянки композиції. Тому введення, повідомлення й результати зв’язуються через стан, а не через ручне оновлення кожного елемента [4].

remember зберігає об’єкт у поточній композиції. rememberSaveable використовує механізм saved instance state для підтримуваних значень. ViewModel відокремлює стан екрана від Activity й переживає зміну конфігурації. Ці механізми мають різні строки життя; жоден сам по собі не є постійною базою даних [4, 5].

Проєкт відкривається в Android Studio з теки PW01. Для синхронізації залежностей і збирання використовується Gradle Wrapper. У таблиці наведено версії з конфігурації проєкту; вони не є характеристиками пристрою, на якому виконувалися перевірки.

Таблиця 2 — Конфігурація проєкту

Параметр	Значення
AGP / Gradle Wrapper	9.4.1 / 9.6.0
Gradle daemon JVM / Java target	JDK 25 / Java 11
Compose compiler plugin / BOM	2.2.10 / 2026.02.01
compileSdk / targetSdk / minSdk	37 / 37 / 26
MPAndroidChart / JUnit	v3.1.0 / 4.13.2

Файл settings.gradle.kts задає репозиторії google(), mavenCentral(), gradlePluginPortal() і JitPack з обмеженням групи com.github.PhilJay. Версії основних залежностей зібрано в gradle/libs.versions.toml. Gradle Wrapper фіксує версію дистрибутива та його SHA-256. Для збирання потрібний повний JDK із засобом jlink; рівень сумісності Java 11 не означає, що процес Gradle запускається на JDK 11.

Таблиця 3 — Структура та відповідальність пакетів

Частина проєкту	Відповідальність
model / logic	Моделі, формули та валідація
data / ui/auth	Акаунти в оперативній пам’яті та стан авторизації
ui/components / ui/screens	Повторно використовувані компоненти й екрани
ui/theme / res	Material-тема, зображення та Android-ресурси
src/test / src/androidTest	Локальні тести / шаблонна перевірка контексту

Рисунок 1 відображає структуру на етапі заняття 2. Під час подальшої роботи додано пакети data, ui/auth та інші компоненти, зазначені в таблиці.

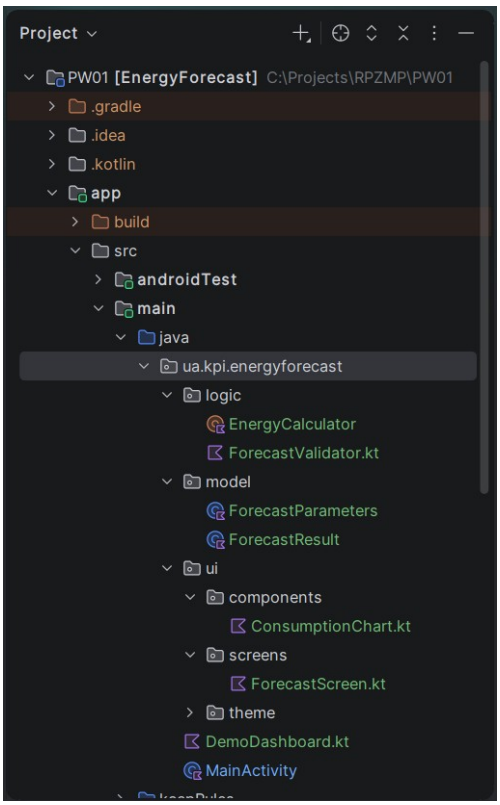


Рисунок 1 — Структура проєкту на етапі практичного заняття 2

3 Практичне заняття 1

На початковому демонстраційному екрані EnergyDashboard відображаються добове споживання, кількість активних приладів і лічильник оновлень. Кнопка «Оновити демодані» генерує нові випадкові показники. Це симуляція динаміки UI; робочий прогноз за введеними параметрами доступний у вкладці «Мої прилади».

На рисунку 2 показано демонстраційний екран на етапі заняття 1. Статичні кадри фіксують зміну показників і лічильника; плавність анімації перевіряється під час роботи застосунку.

Стан dailyConsumption, activeDevices і updateCount зберігається через rememberSaveable. Під час натискання кнопки змінюються цільове споживання й кількість приладів, збільшується лічильник. Для числа споживання застосовано animateFloatAsState з tween тривалістю 900 мс. Анімоване значення форматується до двох знаків із локаллю uk-UA.

Лістинг — DemoDashboard.kt · від рядка 82

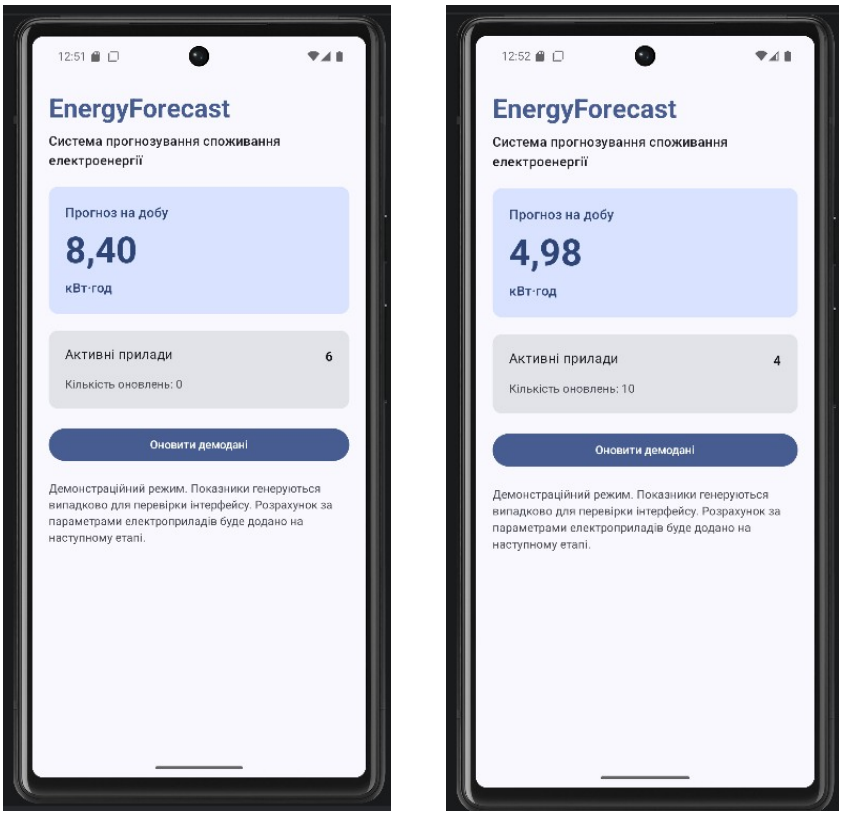
```
// Анімуємо відображення, зберігаючи цільове значення окремо.
val animatedConsumption by animateFloatAsState(
    targetValue = dailyConsumption,
    animationSpec = tween(durationMillis = 900),
    label = "dailyConsumptionAnimation"
)
```

Цільове значення зберігається окремо від проміжних кадрів анімації. Так UI відображає плавний перехід, не змінюючи модель під час кожного кадру. Якщо випадкова генерація поверне те саме споживання, новий перехід числа може бути непомітним, але натискання все одно врахує лічильник.

Лістинг — DemoDashboard.kt · від рядка 110

```
Button(
    onClick = {
        val forecast = generateDemoForecast()

        dailyConsumption = forecast.dailyConsumptionKwh
        activeDevices = forecast.activeDevices
        updateCount += 1
    },
    modifier = Modifier.fillMaxWidth()
) {
    Text(text = "Оновити демодані")
}
```



а б

Рисунок 2 — Оновлення демоданих: а — початковий стан; б — після десяти натискань

Компоненти Column і Row організовують інформацію, Card групує показники, MaterialTheme задає кольори та типографіку. Екран прокручується. Окремі Preview відображають світлу та темну теми.

Для варіанта №1 показники сонячної генерації замінено на споживання електроенергії та кількість електроприладів. Для просунутого рівня реалізовано керування станом, оформлення Material і анімовану реакцію на дію користувача.

4 Практичне заняття 2

Бізнес-логіку винесено в EnergyCalculator та класи перевірки введення. Інтерфейс передає типізовані параметри й відображає ForecastResult. Розрахунок не залежить від Activity, Compose або графічної бібліотеки, тому його можна перевіряти локальними тестами.

Для n груп однакових приладів використовуються формули:

$$Едоб = \Sigma\ (P_i / 1000 \times t_i \times q_i),\ i = 1, \ldots, n \qquad (1)$$

$$Епер = Едоб \times D \qquad (2)$$

P_i — потужність одного приладу, Вт; t_i — години роботи за добу, год; q_i — кількість однакових приладів, шт.; D — період, днів. Едоб вимірюється у кВт·год за добу, Епер — у кВт·год за весь період. Ділення на 1000 переводить вати в кіловати; множення на години дає енергію.

Лістинг — logic/EnergyCalculator.kt · від рядка 39

```
val dailyConsumption =
    parameters.powerWatts / 1000.0 *
            parameters.hoursPerDay *
            parameters.deviceCount

val periodConsumption =
    dailyConsumption * parameters.forecastDays
```

Модель передбачає сталу потужність і однаковий режим роботи щодня. Не враховуються цикли термостата, сезонність або фактичні вимірювання лічильника. Це сценарна розрахункова оцінка без машинного навчання. Кількість приладів враховується для кожної групи, після чого добове споживання груп підсумовується. Результат зберігається у Double без проміжного округлення.

Таблиця 4 — Контрольний розрахунок добового споживання

Прилад	P, Вт	t, год	q	Едоб, кВт·год
Обігрівач	1000	2	1	2,00
Лампи	100	5	3	1,50
Разом	—	—	4	3,50

За 30 днів: 3,50 × 30 = 105,00 кВт·год. Цей приклад перевіряється тестом multipleDevicesAreSummedCorrectly. На рисунку 3 наведено отриманий у застосунку результат; список груп і графік показано на рисунку 7.

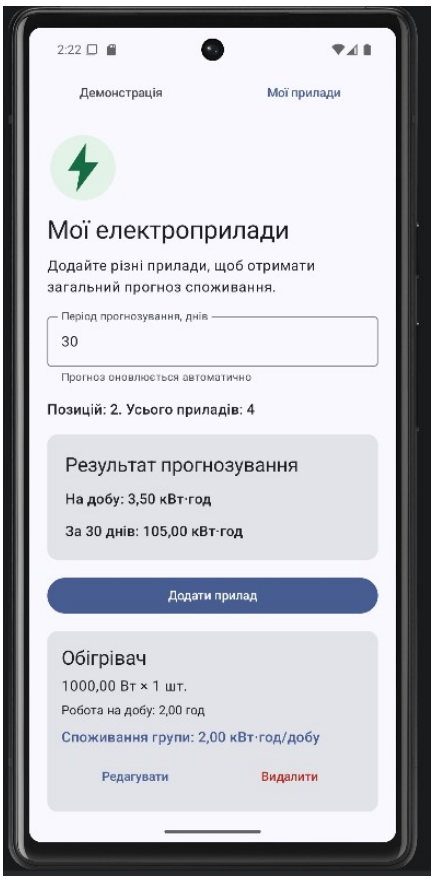


Рисунок 3 — Контрольний приклад: 3,50 кВт·год на добу та 105,00 кВт·год за 30 днів

Для візуалізації в app/build.gradle.kts підключено MPAndroidChart версії 3.1.0 [6]. AndroidView інтегрує LineChart у Compose. Повторно використовуваний компонент ConsumptionChart отримує готовий результат і не дублює формули бізнес-логіки.

```
implementation("com.github.PhilJay:MPAndroidChart:v3.1.0")
```

5 Практичне заняття 3

Робочий екран ApplianceForecastScreen містить зображення energy_illustration, заголовок, поле періоду, підсумок, кнопку додавання, картки DeviceCard і графік. Порожній список має окрему підказку (рисунок 4). Тексти, зображення й кнопки розділені на Composable-функції з українськими коментарями KDoc.

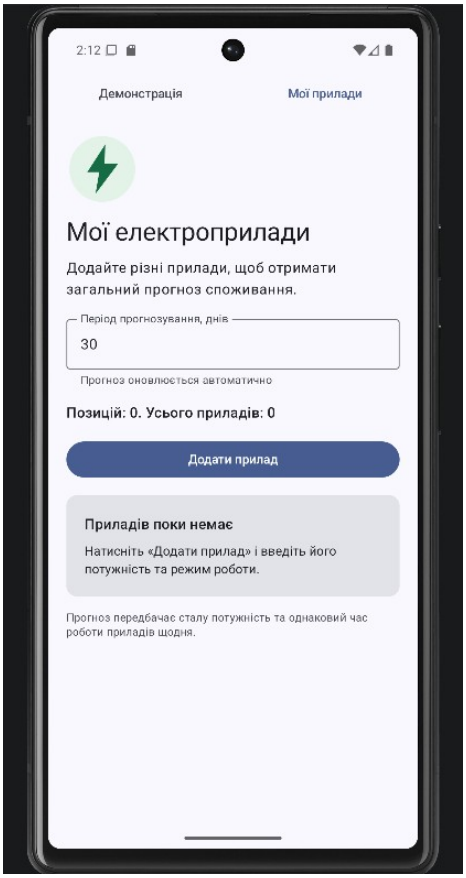
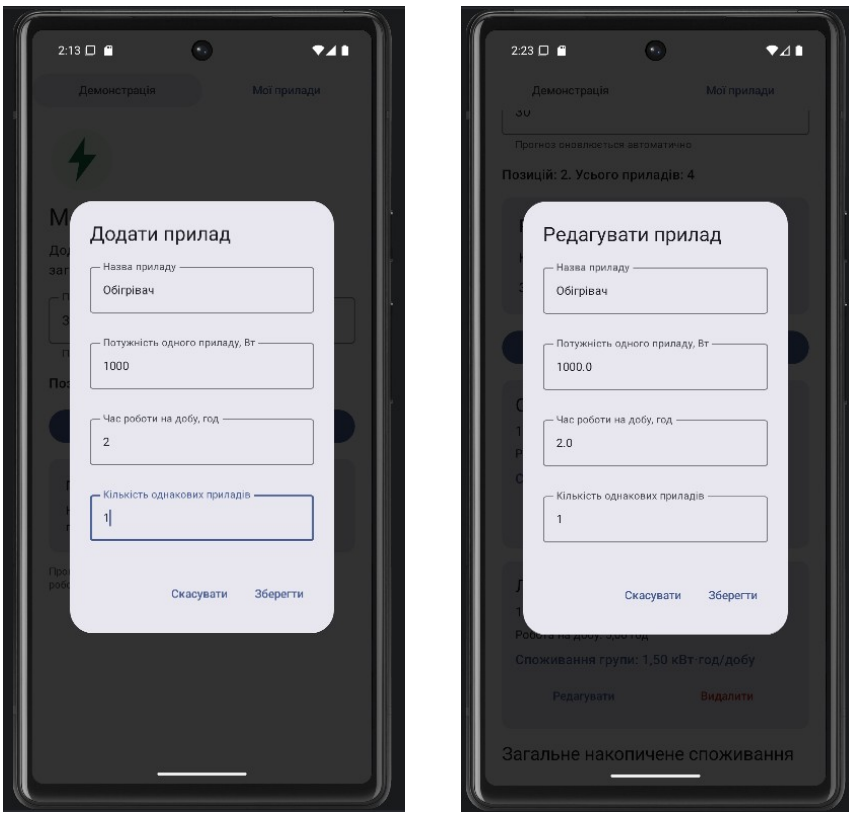


Рисунок 4 — Порожній список приладів на етапі заняття 3

LazyColumn виводить позиції з ключем device.id. Нова позиція отримує UUID; редагування зберігає id. DeviceEditorDialog працює з чернеткою полів: назвою, потужністю одного приладу, часом роботи й кількістю. Лише після успішної перевірки та натискання «Зберегти» об’єкт EnergyDevice передається власнику списку (рисунок 5). Скасування не викликає onSave. Видалення потребує окремого підтвердження (рисунок 6).

Лістинг — ui/components/DeviceEditorDialog.kt · від рядка 132

```
if (parameters != null) {
    onSave(
        EnergyDevice(
            id = device?.id
                ?: UUID.randomUUID().toString(),
            name = validation.name,
            powerWatts = parameters.powerWatts,
            hoursPerDay = parameters.hoursPerDay,
            deviceCount = parameters.deviceCount
        )
    )
}
```



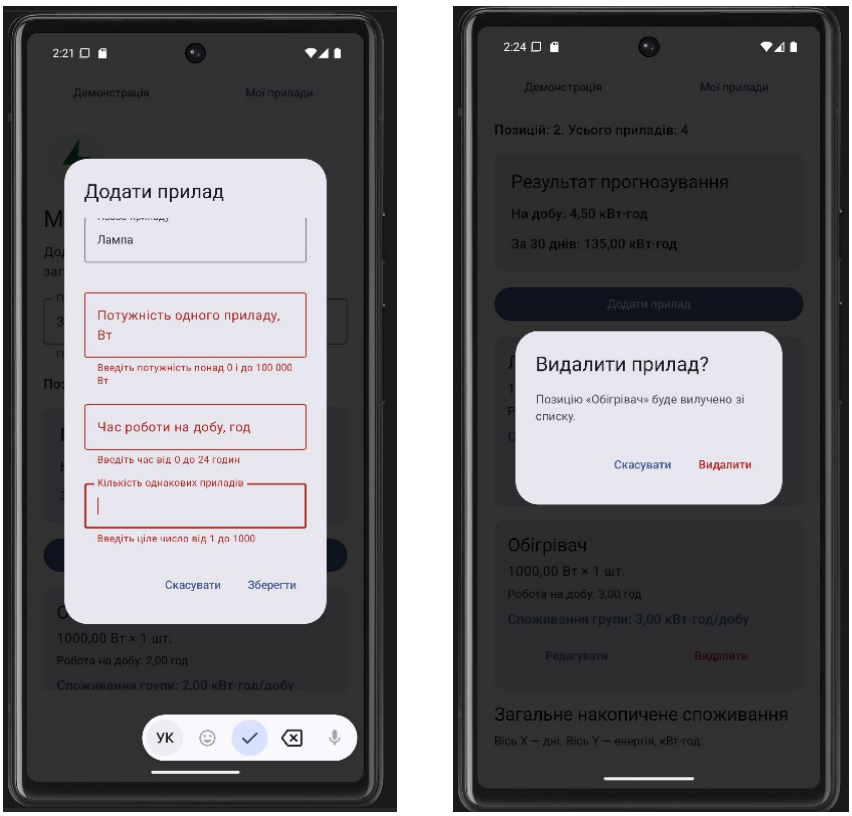
а б

Рисунок 5 — Робота з приладом: а — додавання; б — редагування

Таблиця 5 — Правила введення параметрів

Параметр введення	Правило
Назва приладу	Після trim від 1 до 60 символів
Потужність одного приладу	Скінченна, понад 0 і до 100000 Вт включно
Час роботи на добу	Скінченний, від 0 до 24 год включно
Кількість / період	Цілі: 1–1000 шт. / 1–365 днів
Позиції у списку	До 50 груп приладів

Десяткові числа підтримують кому й крапку. Помилки введення відображаються біля відповідних полів (рисунок 6). Некоректний період приховує результат і графік, щоб не показувати застарілий прогноз. Для адаптивності використано fillMaxWidth, widthIn(max = 840.dp), прокручування, imePadding і adjustResize. Редактор обмежує висоту вмісту та дозволяє його прокручувати. Картки використовують кольори Material, а кнопки мають текстові підписи.



а б

Рисунок 6 — Обробка дій: а — помилки введення; б — підтвердження видалення

6 Стан та графік споживання

Список і параметри екрана зберігаються через rememberSaveable [4]. DeviceListSaver записує для кожної позиції п’ять підтримуваних значень у визначеному порядку: id, name, powerWatts, hoursPerDay, deviceCount. Під час відновлення список значень ділиться на групи по п’ять елементів, з яких реконструюються об’єкти EnergyDevice зі збереженням їхніх ідентифікаторів.

Лістинг — ui/screens/ApplianceForecastScreen.kt · від рядка 91

```
val result = remember(devices, forecastDays) {
    if (devices.isNotEmpty() && forecastDays != null) {
        EnergyCalculator.calculateTotal(
            devices = devices,
            forecastDays = forecastDays
        )
    } else {
        null
    }
}
```

Прогноз є похідним від списку та періоду, тому окремо зберігати результат не потрібно. Зміна devices або forecastDays запускає повторний розрахунок. MainScreen зберігає стани вкладок через SaveableStateHolder; MainActivity використовує зовнішній SaveableStateHolder із ключем user.id для відокремлення станів акаунтів.

Таблиця 6 — Життєвий цикл стану

Подія	Очікувана поведінка за кодом
Повторна композиція	Значення remember зберігаються, доки відповідний вузол лишається в композиції
Поворот Activity	ViewModel зберігається, saved UI state відновлюється
Перемикання вкладок	Стани вкладок утримуються за їхніми ключами
Інший акаунт у сеансі	Окремий стан інтерфейсу за user.id
Завершення процесу системою	Акаунти й сесія втрачаються; відновлення UI залежить від saved instance state
Закриття застосунку	Постійне збереження даних не гарантується

Графік накопичення містить точки від дня 0 до D: E(k) = Едоб × k, де k — номер дня. Вісь X показує дні, вісь Y — накопичену енергію у кВт·год. За незмінного режиму графік лінійний і для контрольного прикладу завершується значенням 105,00 кВт·год (рисунок 7). Перетворення Double на Float виконується лише для графічної бібліотеки. Кольори беруться з MaterialTheme; contentDescription описує підсумок для засобів доступності.

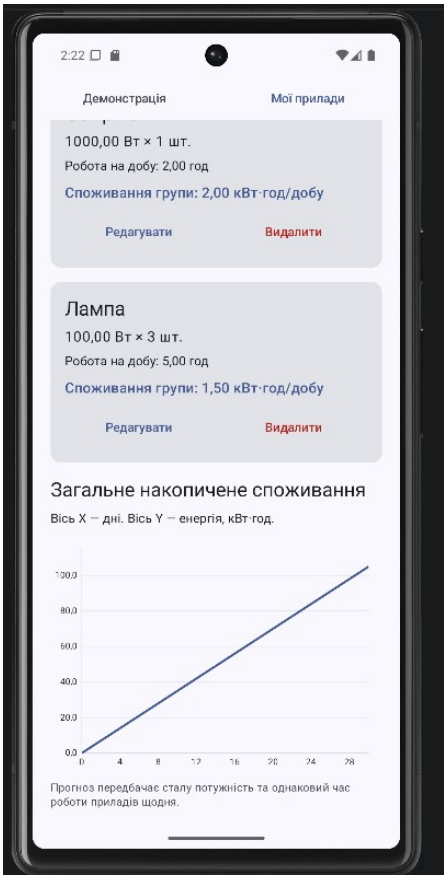


Рисунок 7 — Список груп і графік накопичення для контрольного прикладу

Рисунок 8 демонструє горизонтальне компонування на етапі розроблення. Тут використано інший набір параметрів: 4,50 кВт·год на добу і 135,00 кВт·год за 30 днів. Цей кадр не належить до контрольного прикладу.

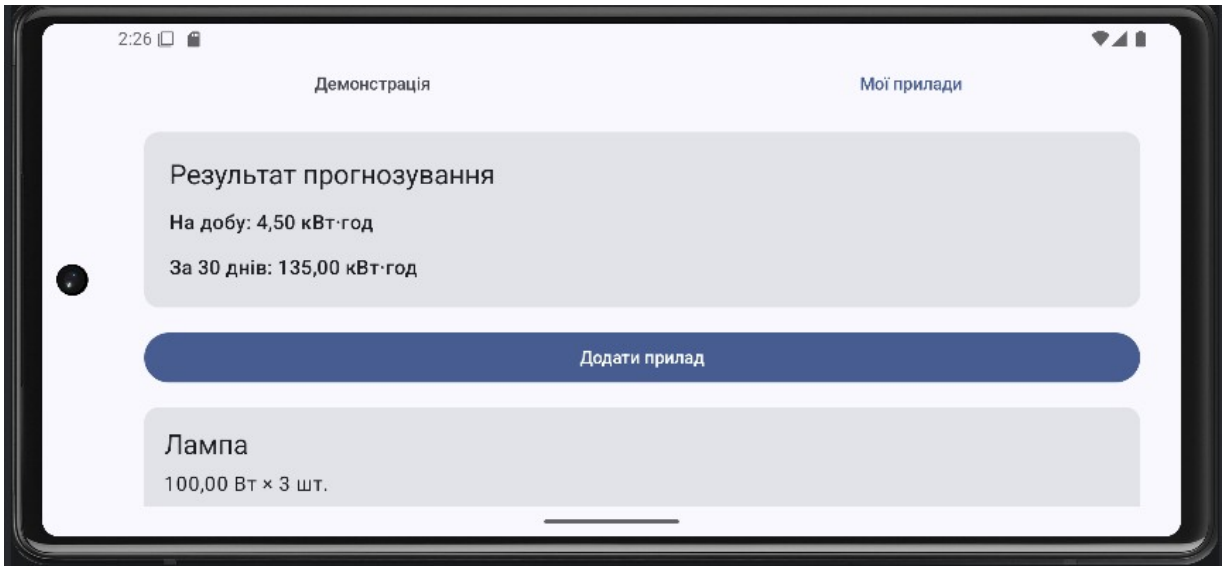


Рисунок 8 — Горизонтальне компонування екрана з іншим набором параметрів

Темну тему фінального інтерфейсу наведено на рисунку 9. Скриншоти показують окремі стани; збереження введення після повороту підтверджується ручною перевіркою, описаною в розділі 9.

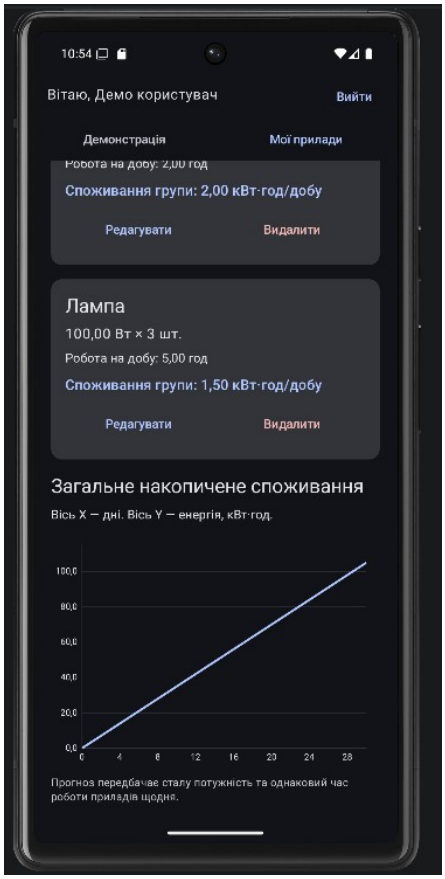


Рисунок 9 — Список і графік у темній темі

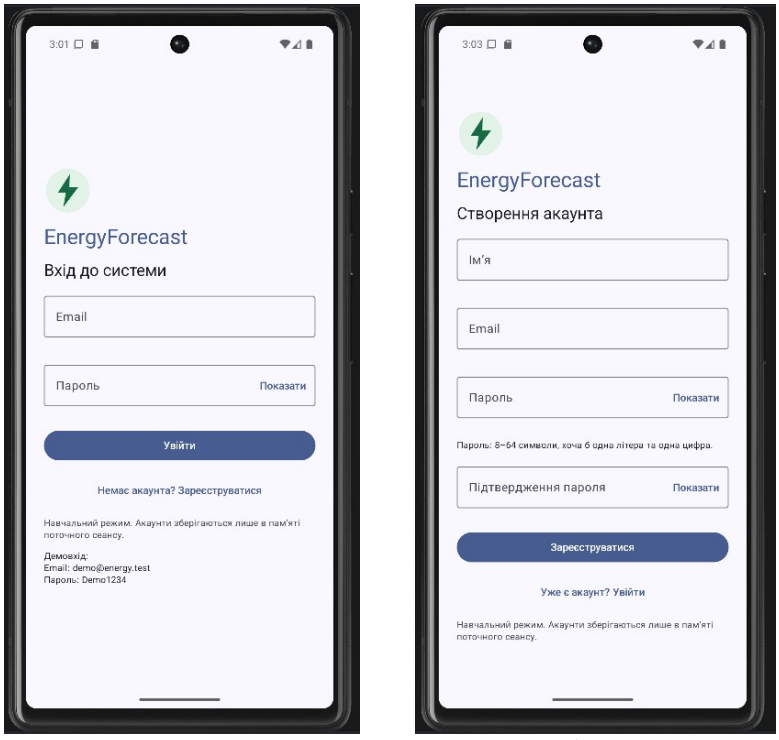
Відновлення UI не тотожне базі даних. Після втрати процесу створені акаунти недоступні. Повторна реєстрація такого самого email створить новий id, тому вона не відновлює попередній профіль.

7 Практичне заняття 4

AuthScreen перемикає форми входу та реєстрації (рисунок 10). AuthHeader, AuthTextField, AuthPasswordField і AuthMessage відокремлюють заголовок, поля та повідомлення. Значення надходять з AuthUiState, а функції зворотного виклику передають дії до AuthViewModel. Видимість пароля керується локально, тоді як сам пароль належить стану форми.

Таблиця 7 — Правила форм авторизації

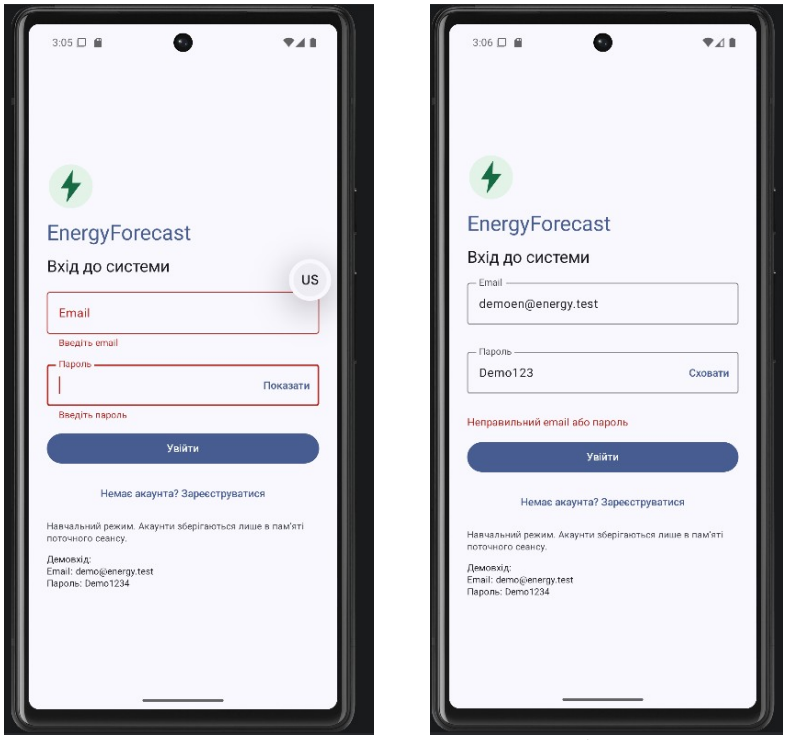
Поле або операція	Правило
Ім’я під час реєстрації	2–60 символів після trim
Email	Після trim до 254 символів, без пробілів, із @ і крапкою в домені
Пароль реєстрації	8–64 символи; хоча б одна літера й цифра
Підтвердження	Непорожнє, точний збіг із паролем
Вхід	Коректний email і непорожній пароль; точна перевірка у сховищі акаунтів
Нормалізація акаунта	trim().lowercase(Locale.ROOT); дублі email заборонено



а б

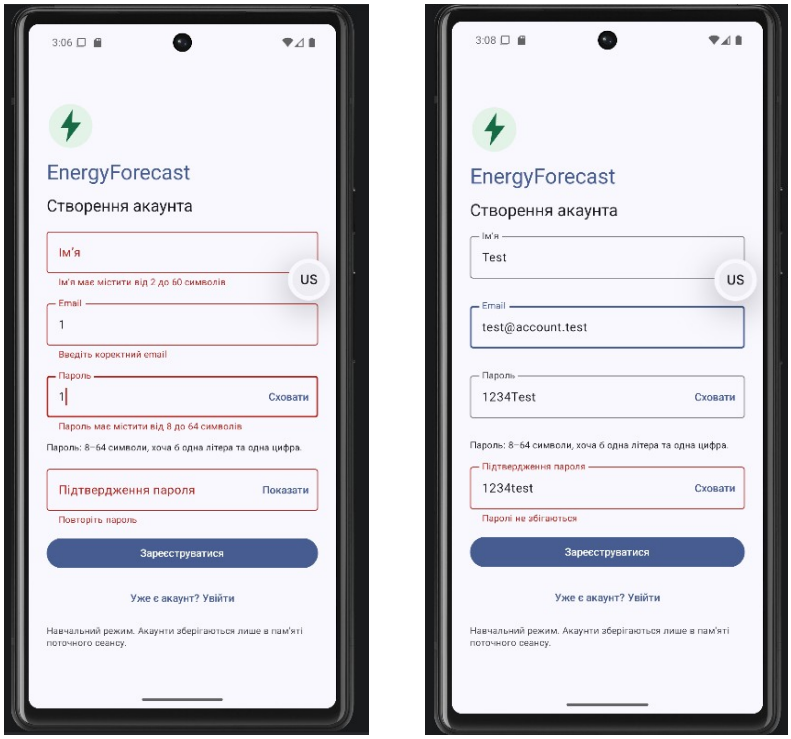
Рисунок 10 — Форми авторизації: а — вхід; б — реєстрація

Метод submit спочатку фіксує поточні значення форми та перевіряє їх. Помилки зупиняють операцію до переходу в стан обробки (рисунки 11, 12). isLoading встановлюється синхронно перед запуском корутини, тому повторне натискання не створює ще одну операцію. Поля та перемикання режиму в цей час заблоковані. Затримка 1200 мс імітує запит, не блокуючи головний потік (рисунок 13).



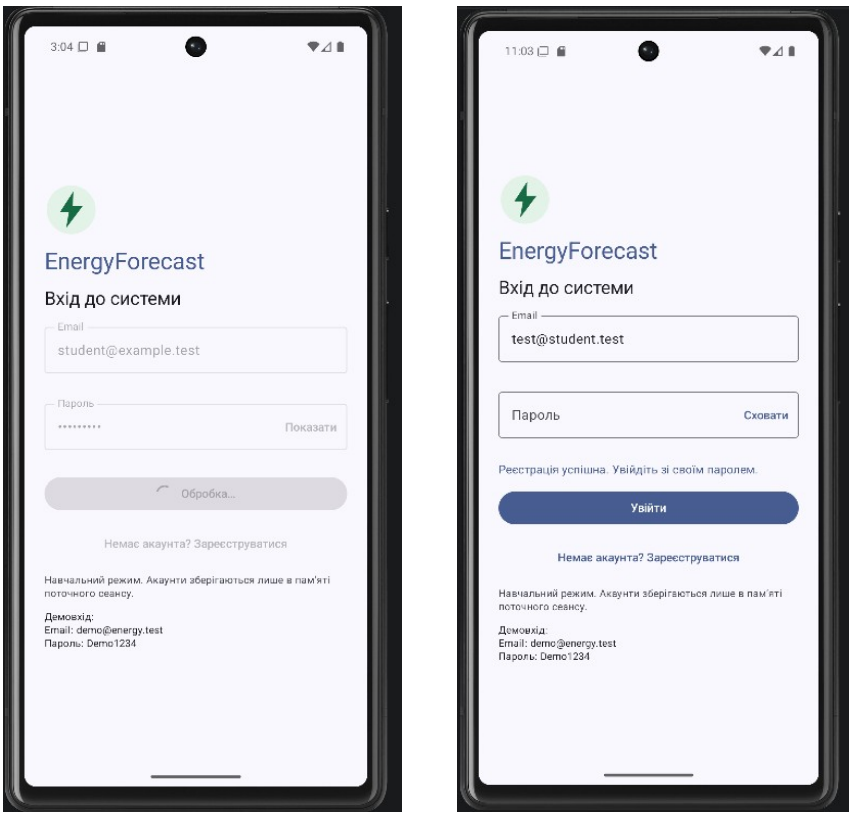
а б

Рисунок 11 — Помилки входу: а — порожні поля; б — відхилені облікові дані



а б

Рисунок 12 — Помилки реєстрації: а — некоректні значення полів; б — різні паролі



а б

Рисунок 13 — Стани обробки: а — очікування із заблокованою формою; б — результат успішної реєстрації

Лістинг — фрагмент submit з ui/auth/AuthViewModel.kt

```
if (errors.isNotEmpty()) return
uiState = uiState.copy(isLoading = true)

viewModelScope.launch {
    try {
        delay(1200)
```

Успішна реєстрація повертає до входу з email і повідомленням, а пароль очищується. Успішний вхід встановлює user і відкриває основний екран. Невдалий вхід завершує обробку та показує помилку. Після редагування невалідної форми актуальні помилки інших полів зберігаються. Вихід очищує стан авторизації, але сховище акаунтів існує до знищення ViewModel.

DemoAuthRepository зберігає акаунти й паролі у відкритому вигляді лише в оперативній пам’яті. Це навчальна реалізація без сервера, токенів і захисту паролів, потрібного для реальної системи. Для демонстрації використовуються вигадані дані та навчальні паролі.

AuthUiState об’єднує режим форми, ім’я, email, пароль і його підтвердження, помилки полів та ознаку обробки isLoading. Поля message і messageIsError задають повідомлення та його тип, а user містить поточного користувача або null. AuthViewModel оновлює цей стан, і Compose відображає відповідну форму, індикатор або основний екран; ViewModel зберігає стан під час зміни конфігурації [5].

8 Автоматичне тестування

У вихідних файлах перевіреної версії міститься 38 локальних методів з анотацією @Test. Цикл із кількома наборами даних враховується як один метод. Окремий ExampleInstrumentedTest перевіряє назву пакета, а не сценарії інтерфейсу; його запуск до наведеного підсумку не входить. Усі 24 власні Composable-функції мають KDoc. HTML-підсумок локального запуску наведено на рисунку 14.

Таблиця 8 — Склад локальних автоматичних тестів

Клас	Методів	Предмет перевірки
EnergyCalculatorTest	10	Формули, кількість, суми, нулі, NaN, нескінченність і межі
ForecastValidatorTest	7	Кома, помилки, межі, Int overflow
AuthValidatorTest	8	Email, пароль, підтвердження й довжини
AuthViewModelTest	6	Стан обробки, помилки полів, повторне надсилання, переходи
DemoAuthRepositoryTest	4	Вхід, неправильний пароль, нормалізація, дублі
DeviceValidatorTest	2	Назва та trim
ExampleUnitTest	1	Шаблонне додавання 2 + 2

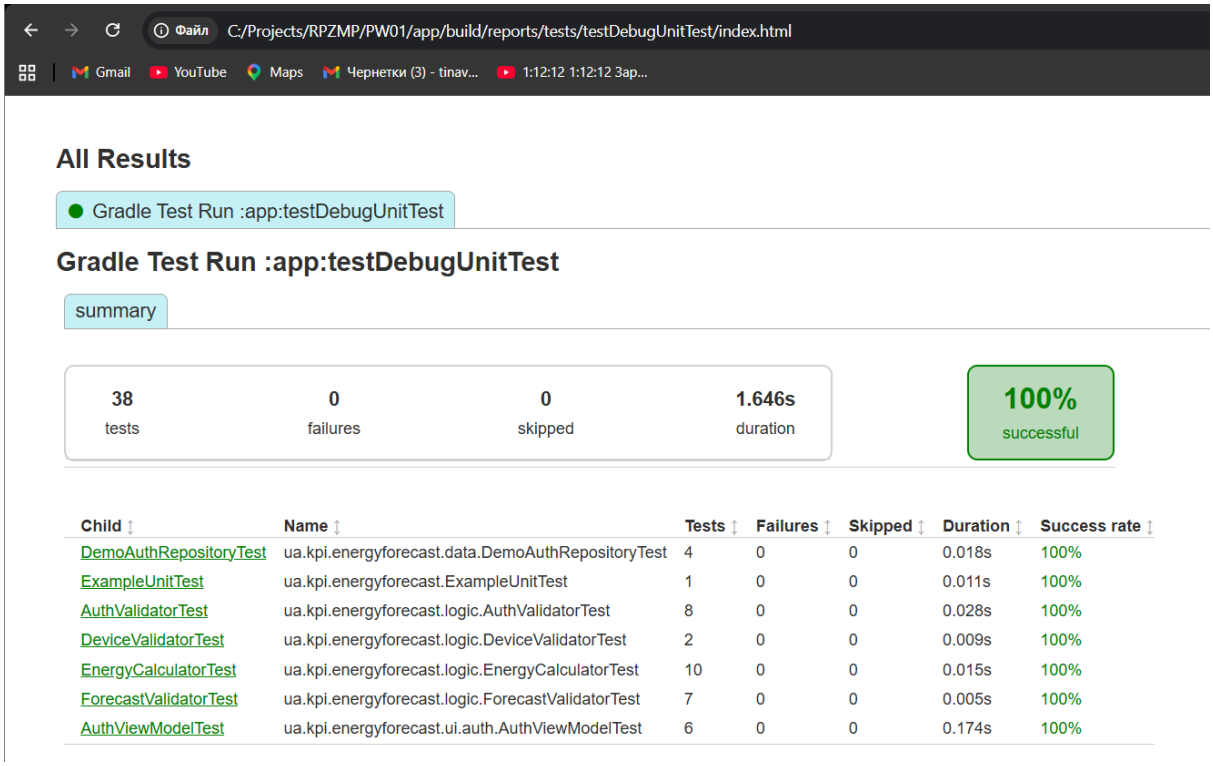


Рисунок 14 — HTML-підсумок локальних автоматичних тестів

Лістинг — EnergyCalculatorTest.kt, граничний сценарій

```
@Test
fun maximumUiInputsRemainFiniteAndIncludeAllQuantities() {
    val devices = List(50) { index ->
        EnergyDevice("device-$index", "Прилад", 100000.0, 24.0, 1000)
    }

    val result = EnergyCalculator.calculateTotal(devices, 365)

    assertEquals(120000000.0, result.dailyConsumptionKwh, 0.000001)
    assertEquals(43800000000.0, result.periodConsumptionKwh, 0.000001)
}
```

Максимальна конфігурація інтерфейсу — 50 позицій по 100000 Вт, 24 год і 1000 одиниць — дає 120000000 кВт·год на добу та 43800000000 кВт·год за 365 днів. Тест перевіряє підсумок з урахуванням кількості без переповнення. Окремі тести перевіряють відхилення NaN, ±Infinity та нескінченного результату зі скінченних аргументів. Це граничні значення програми, а не типовий побутовий сценарій.

Тести авторизації перевіряють довжини пароля 7/8/64/65, імені 1/2/60/61 після trim та email 254/255. AuthViewModelTest використовує віртуальний час для перевірки стану обробки й захисту від повторної реєстрації. Повні сценарії додавання, редагування й видалення приладів, повороту, перемикання акаунтів і доступності не охоплені автоматичними UI-тестами.

9 Результати та джерела перевірки

Локальну перевірку виконано 29.10.2026. HTML-підсумок підтверджує 38 тестів, 0 помилок і 0 пропусків (рисунок 14). Огляд Lint-звіту містить 22 попередження та 3 підказки (рисунок 15). Успішне виконання testDebugUnitTest, lintDebug та assembleDebug зафіксовано за результатами локальної перевірки авторки роботи. Фрагмент термінала на рисунку 16 показує BUILD SUCCESSFUL без самої команди запуску.

Таблиця 9 — Підсумок перевірок та джерела доказів

Перевірка	Результат	Доказ
Локальні тести	38 тестів; 0 помилок; 0 пропусків	HTML-підсумок, рисунок 14
Android Lint	22 попередження та 3 підказки	Огляд звіту, рисунок 15
testDebugUnitTest, lintDebug, assembleDebug	Успішне виконання	Повідомлення авторки; рисунок 16 показує лише завершення збірки
Ручні сценарії	Пройдено за повідомленням авторки	Таблиця сценаріїв і скриншоти окремих станів
Статичний огляд коду	38 локальних @Test; 24 власні Composable з KDoc	Вихідні файли PW01 зазначеної версії

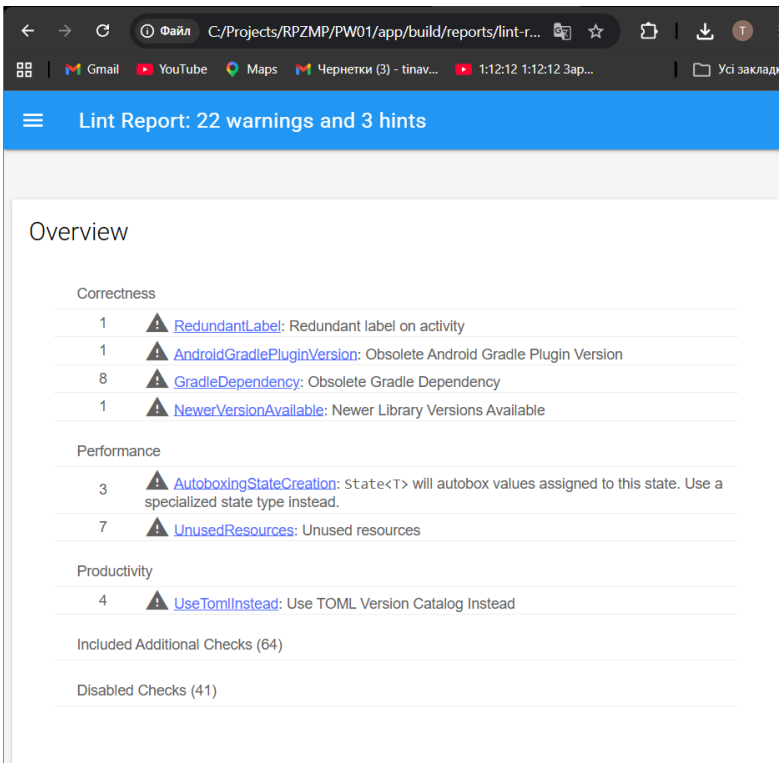


Рисунок 15 — Огляд Android Lint: 22 попередження та 3 підказки

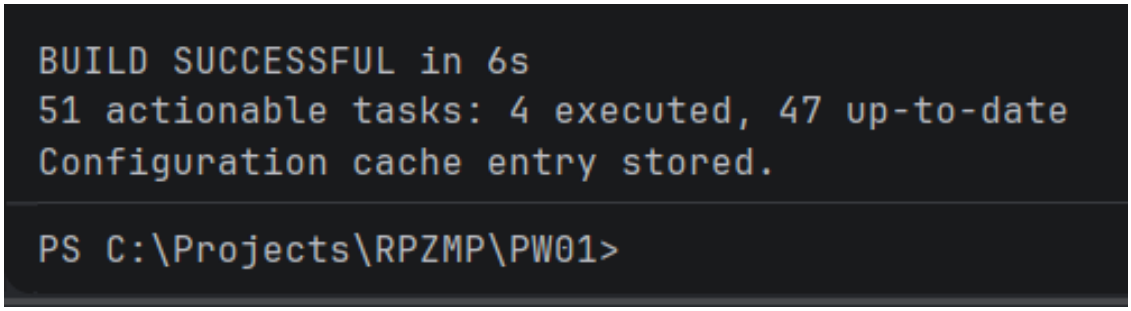


Рисунок 16 — Фрагмент термінала з повідомленням BUILD SUCCESSFUL

На оглядовому екрані Lint видно зауваження щодо версій залежностей, невикористаних ресурсів, створення стану з упакованим числовим типом, каталогу версій та назви Activity. Повний HTML-звіт із деталями кожного зауваження не досліджувався, тому цей огляд не визначає їхній вплив на всі сценарії. Пристрій і версії Android та Android Studio не зафіксовані; скриншоти не встановлюють прив’язку локального запуску до commit. Ці обмеження враховано під час інтерпретації результатів.

Для відтворення перевірок з теки PW01 слід виконати команди нижче. Підсумки доступні у файлах app/build/reports/tests/testDebugUnitTest/index.html та app/build/reports/lint-results-debug.html.

```
.\gradlew.bat :app:testDebugUnitTest --no-daemon
.\gradlew.bat :app:lintDebug --no-daemon
.\gradlew.bat :app:assembleDebug --no-daemon
```

Таблиця 10 — Сценарії ручної перевірки

Сценарій	Очікуваний результат	Результат ручної перевірки
Контрольний приклад	3,50 кВт·год/добу; 105,00 кВт·год за 30 днів	Збіг підтверджено
Валідація	Помилки й відсутність невалідного прогнозу	Пройдено
CRUD і скасування	Зміна лише після підтвердження	Пройдено
Login, register, loading, logout	Коректні переходи та повідомлення	Пройдено
Графік	Накопичення за обраний період	Пройдено
Поворот, вкладки, акаунти	Відновлення та розділення станів	Пройдено
Зручність UI	Читабельність і доступне керування	Пройдено в перевіреному середовищі

10 Висновки

У межах практичної роботи розроблено Android-застосунок EnergyForecast для варіанта №1. Послідовно реалізовано динамічний демонстраційний екран з анімацією, відокремлені моделі й обчислення, керування списком електроприладів та навчальні форми входу й реєстрації. Прийоми прикладу варіанта №30 адаптовано до розрахунку споживання за потужністю, тривалістю роботи та кількістю приладів.

Контрольний розрахунок дає 3,50 кВт·год на добу та 105,00 кВт·год за 30 днів. Калькулятор підсумовує енергію груп без проміжного округлення; класи валідації перевіряють межі й відхиляють NaN та нескінченні значення. Результат і накопичувальний графік залежать від актуального стану списку. Стабільні ідентифікатори дозволяють редагувати та видаляти конкретні позиції.

Під час роботи опрацьовано життєвий цикл стану. ViewModel зберігає стан авторизації під час повороту, механізм saved instance state підтримує відновлення введення, а SaveableStateHolder розділяє стани вкладок і акаунтів. Ці рішення не забезпечують постійного збереження. Розрахунок передбачає незмінну потужність і однаковий режим роботи щодня; авторизація використовує лише оперативну пам’ять поточного сеансу.

HTML-підсумок засвідчує проходження 38 локальних тестів без помилок і пропусків. За результатами локальної перевірки успішно виконано testDebugUnitTest, lintDebug та assembleDebug; Lint містить 22 попередження та 3 підказки. Ручні сценарії перевірено авторкою роботи. Отриманий застосунок поєднує функціональність чотирьох практичних занять: динамічний інтерфейс, відокремлені обчислення, компоненти роботи з приладами та форми авторизації.

11 Використані джерела

[1] Розробка програмного забезпечення мобільних пристроїв. Практична робота №1, практичні заняття 1–4. КПІ ім. Ігоря Сікорського, 2026. Наданий файл PW_01 RPZMP (2) (1).pdf. Вимоги: с. 11–12, 20–21, 30–31, 40–41; варіанти: с. 126.

[2] Розробка програмного забезпечення мобільних пристроїв : навч. посіб. / уклад.: О. Л. Недашківський, І. І. Гусєва, В. О. Голець, М. С. Гнатишин, Т. В. Пироговська, В. Р. Передера. Київ : КПІ ім. Ігоря Сікорського, 2025. 316 с. URL: https://ela.kpi.ua/handle/123456789/74698 (дата звернення: 29.10.2026).

[3] Розробка програмного забезпечення мобільних пристроїв. Навчально-методичний комплекс : навч. посіб. / уклад.: О. Л. Недашківський, М. А. Черниш. Київ : КПІ ім. Ігоря Сікорського, 2026. 1303 с. URL: https://ela.kpi.ua/handle/123456789/82805 (дата звернення: 29.10.2026).

[4] State and Jetpack Compose // Android Developers. URL: https://developer.android.com/develop/ui/compose/state (дата звернення: 29.10.2026).

[5] ViewModel overview // Android Developers. URL: https://developer.android.com/topic/libraries/architecture/viewmodel (дата звернення: 29.10.2026).

[6] MPAndroidChart. Version 3.1.0 // GitHub. URL: https://github.com/PhilJay/MPAndroidChart/releases/tag/v3.1.0 (дата звернення: 29.10.2026).